Curvature-Velocity Scheduled LPV H_∞ Path-Following Controller Design via Vector Field Guidance

Suwon Lee[1*†]

[1*]Department of Future Mobility, Kookmin University, Seoul, South Korea.

Corresponding author(s). E-mail(s): suwon.lee@kookmin.ac.kr;
[†]ORCID: 0000-0002-6573-6348

Abstract

Vector field guidance (VFG) generates a desired heading whose rate of change is governed by the product of path curvature and vehicle speed. This coupling introduces a speed- and geometry-dependent guidance bandwidth that conventional fixed-gain controllers cannot accommodate across varying operating conditions. This paper proposes a curvature-velocity scheduled linear parameter-varying (LPV) $\boldsymbol{H_\infty}$ controller that adapts its performance weighting bandwidth in real time through the scheduling parameter $\boldsymbol{\rho = \kappa v}$. A concave square-root schedule distributes bandwidth across multiple polytopic vertices synthesized offline, and the runtime controller is obtained by convex output interpolation at 11 $\boldsymbol{\mu}$s per step—well within real-time constraints and far less demanding than linear MPC. Frequency-domain analysis confirms that all vertices satisfy classical sensitivity robustness bounds, while time-domain simulations on a differential-drive scale car demonstrate substantial tracking improvement over linear MPC and PID with curvature feedforward, particularly at moderate to high speeds. The proposed controller exhibits near speed-invariant heading error across the tested speed range, in contrast to the monotonic performance degradation observed in the baselines. Monte Carlo analysis under multi-dimensional parametric uncertainty further confirms robust stability, statistically significant performance gains at moderate-to-high speeds, and faster transient settling across all tested speeds.

Keywords: Path following, Vector field guidance, LPV control, $\boldsymbol{H_\infty}$ synthesis, Gain scheduling, Autonomous vehicles

1 Introduction

Path following is a fundamental capability for autonomous ground vehicles and mobile robots (Aguiar and Hespanha, 2007; Snider, 2009). Vector field guidance (VFG) offers a geometrically intuitive approach to this problem. It constructs a smooth velocity field that steers the vehicle toward the reference path by combining a convergence component with a tangential component (Gonçalves et al, 2010; Sujit et al, 2014). Because VFG decouples the guidance logic from the low-level steering controller, it has been adopted in UAV navigation (Nelson et al, 2007) and ground vehicle applications alike.

However, VFG only prescribes a desired heading ψ_{des}; a separate controller must translate this heading into a steering command δ. The effective guidance bandwidth is $\omega_{\text{guid}} = \kappa v$, where κ is the path curvature and v is the vehicle speed. On a sharp curve at high speed, ω_{guid} is large and the controller must track rapidly. On a straight segment, ω_{guid} drops to zero and sensor noise dominates. A fixed-gain controller cannot satisfy both demands simultaneously. This *bandwidth mismatch* is the central problem addressed in this paper.

1.1 Related Work

Nelson et al (2007) developed vector field path following for miniature air vehicles with wind rejection. Kapitanyuk et al (2018) proposed a guiding vector field algorithm with singularity-free guarantees for nonholonomic mobile robots. Both works assume the steering controller has adequate bandwidth across all operating conditions, an assumption that breaks down when curvature and speed vary widely.

Linear parameter-varying (LPV) control provides a systematic framework for gain scheduling with stability guarantees. Apkarian et al (1995) established the self-scheduled H_∞ design methodology. In the vehicle domain, Corno et al (2021) scheduled the plant matrices by longitudinal speed v_x while keeping the performance specification fixed. Kapsalis et al (2022) employed curvature-based scheduling in a polytopic LPV look-ahead controller but without H_∞ optimization. Gáspár et al (2005) applied LPV methods to integrated yaw and roll dynamics. Sename et al (2024) surveyed robust and predictive control strategies for LPV vehicle systems. These approaches schedule either the plant model or the controller gains, but none of them schedule the *performance specification* by a parameter that captures the guidance bandwidth.

On the baseline side, model predictive control (MPC) is widely used for vehicle path following. Falcone et al (2007) developed predictive active steering for autonomous vehicles. PID with curvature feedforward remains a practical baseline due to its simplicity (Snider, 2009). The bicycle model (Rajamani, 2012) is the standard plant for lateral control design, and loop-shaping principles from Skogestad and Postlethwaite (2005) inform the weighting function selection.

1.2 Contributions

This paper proposes scheduling the performance weight bandwidth by $\rho = \kappa v$, a parameter that arises naturally from the on-path heading dynamics of the VFG formulation. Unlike existing methods that schedule the plant model by speed (Corno et al,

2021) or the controller by curvature (Kapsalis et al, 2022), the proposed approach directly adapts the H_∞ performance specification to match the guidance bandwidth. The contributions are as follows:

1. **Guidance bandwidth characterization.** We show that the product $\rho = \kappa v$ governs the on-path heading dynamics, providing a physics-based justification for using ρ as the scheduling variable.
2. **ρ-scheduled LPV H_∞ controller.** A polytopic LPV H_∞ controller is synthesized at six vertices with a square-root bandwidth schedule $\omega_B(\rho)$. Frequency-domain analysis confirms that the peak sensitivity remains below 6 dB at every vertex, and the runtime controller requires only two matrix-vector multiplications per step (approximately 11 μs), making it suitable for embedded deployment.
3. **Speed-invariant performance with statistical validation.** The proposed controller is compared against linear MPC and PID with curvature feedforward on step curvature and slalom paths at speeds from 0.5 to 3.0 m/s. A Speed Invariance index quantifies the nearly flat error profile of the LPV controller across speeds. Monte Carlo analysis with 200 Latin Hypercube Sampling (McKay et al, 1979) under five-dimensional parametric uncertainty confirms 100% stability and statistically significant improvement (Wilcoxon $p < 10^{-10}$) at moderate-to-high speeds.

1.3 Paper Organization

The remainder of this paper is organized as follows. Section 2 analyzes the VFG formulation and derives the scheduling parameter ρ. Section 3 presents the vehicle model and uncertainty structure. Section 4 details the LPV H_∞ controller design. Section 5 presents the simulation results. Section 6 concludes the paper.

2 Vector Field Guidance

2.1 VFG Formulation

Consider a reference path $\mathcal{P} = \{\mathbf{p}(s) : s \in [0, s_f]\}$ parameterized by arc length s. At vehicle position $\mathbf{q} = [x,\, y]^\top$, let s^* be the arc length of the closest point on $\mathcal{P}$ and let $d = \|\mathbf{q} - \mathbf{p}(s^*)\|$ denote the lateral distance. VFG (Nelson et al, 2007; Gonçalves et al, 2010) assigns a desired velocity direction by blending a convergence component toward the path with a tangential component along it. Although the original formulation was developed for implicit curves $f(x, y) = 0$, VFG extends naturally to parametric curves via signed-distance construction (Lee, 2023). The desired velocity is

$$\mathbf{v}_{\text{des}} = v_d\big[k_{\text{conv}}(d)\,\hat{\mathbf{v}}_{\text{conv}} + k_{\text{trav}}(d)\,\mathbf{t}(s^*)\big], \tag{1}$$

where v_d is the desired speed, $\mathbf{t}(s) = d\mathbf{p}/ds$ is the unit tangent, and $\hat{\mathbf{v}}_{\text{conv}}$ is the unit vector from $\mathbf{q}$ toward $\mathbf{p}(s^*)$. The weighting functions satisfy $k_{\text{conv}}^2 + k_{\text{trav}}^2 = 1$ via the smooth parameterization

$$k_{\text{conv}}(d) = \tanh(k_e\, d), \quad k_{\text{trav}}(d) = \sqrt{1 - k_{\text{conv}}^2(d)}, \tag{2}$$

with convergence gain $k_e > 0$. Far from the path, $k_{\text{conv}} \to 1$ and the field steers toward $\mathcal{P}$; near the path, $k_{\text{trav}} \to 1$ and the field aligns with $\mathbf{t}$.

2.2 Guidance Bandwidth

The desired heading $\psi_{\text{des}} = \angle \mathbf{v}_{\text{des}}$ serves as the reference signal for the steering controller. When the vehicle is on the path ($d \approx 0$), $k_{\text{trav}} \approx 1$ and the desired heading reduces to the path tangent angle: $\psi_{\text{des}} \approx \psi_{\text{path}}(s)$. Differentiating with respect to time gives

$$\dot{\psi}_{\text{des}} = \frac{d\psi_{\text{path}}}{ds}\frac{ds}{dt} = \kappa\, v,$$

where $\kappa = \kappa(s^*)$ is the path curvature and $v = ds/dt$ is the longitudinal speed. The product $\omega_{\text{guid}} = \kappa v$ therefore represents the rate at which the guidance command changes, i.e., its bandwidth. We define the scheduling variable

$$\rho \triangleq \kappa\, v$$

to capture this coupled curvature–speed effect. On a straight segment ($\kappa = 0$), $\rho = 0$ and the guidance command is essentially constant; on a tight curve at high speed, ρ is large and the reference changes rapidly. A fixed-bandwidth controller faces a fundamental trade-off: high bandwidth tracks the fast transient during curves but amplifies sensor noise on straights, and vice versa. For example, on a step curvature path ($\kappa = 2\,\text{m}^{-1}$) at $v = 2\,\text{m/s}$, the guidance bandwidth jumps instantaneously from 0 to 4 rad/s at the curvature onset. This motivates a controller whose bandwidth adapts with ρ.

3 Vehicle Model

3.1 Kinematic Bicycle Model

The controller design relies on a kinematic bicycle model, which accurately captures the heading dynamics of a differential-drive scale car at moderate speeds. Linearizing about a constant longitudinal speed v_0 and zero steering angle yields the state-space form (Rajamani, 2012):

$$\begin{bmatrix}\dot{\psi}\\ \dot{\delta}\end{bmatrix} = \begin{bmatrix}0 & v_0/L\\ 0 & -1/\tau_\delta\end{bmatrix}\begin{bmatrix}\psi\\ \delta\end{bmatrix} + \begin{bmatrix}0\\ 1/\tau_\delta\end{bmatrix}\delta_{\text{cmd}}, \tag{3}$$

where $L = 0.2\,\text{m}$ is the wheelbase and $\tau_\delta = 0.07\,\text{s}$ is the steering actuator time constant. The state $[\psi, \delta]^\top$ consists of the vehicle heading and the front steering angle; δ_{cmd} is the commanded steering input. Table 1 lists the scale-car platform parameters used throughout this study.

3.2 Dynamic Bicycle Model

To validate robustness beyond the kinematic regime, the Monte Carlo simulations in Sec. 5 employ a dynamic bicycle model. The augmented state $\mathbf{x} = [\psi,\, v_y,\, r,\, \delta]^\top$

Table 1 Scale Car Parameters

Parameter	Symbol	Nominal	Unit
Wheelbase	L	0.200	m
Mass	m	4.8	kg
Yaw inertia	I_z	0.15	kg·m^2
Front cornering stiffness	$C_{\alpha f}$	20	N/rad
Rear cornering stiffness	$C_{\alpha r}$	20	N/rad
Steering time constant	τ_δ	0.07	s
Front/rear CG distance	l_f, l_r	0.10	m

evolves as

$$\dot{\mathbf{x}} = \mathbf{A}_d(v_x)\,\mathbf{x} + \mathbf{B}_d\,\delta_{\text{cmd}}, \tag{4}$$

where v_y is the lateral velocity and $r = \dot{\psi}$ is the yaw rate. The system matrix $\mathbf{A}_d$ depends on v_x through the linearized tire slip angles, making the plant inherently parameter-varying.

3.3 Noise and Uncertainty

The measured outputs are the heading angle $\psi_{\text{meas}} = \psi + n_\psi$ and the steering angle $\delta_{\text{meas}} = \delta + n_\delta$. The heading noise n_ψ ($\sigma_\psi = 0.02$–$0.05\,\text{rad}$) reflects Lidar-based localization accuracy, while the encoder noise n_δ ($\sigma_\delta = 0.01$–$0.02\,\text{rad}$) represents steering sensor resolution. These noise levels are incorporated into the weighting function design in Sec. 4.

To assess robustness against plant–model mismatch, three parametric uncertainties are considered: mass variation $m \in [4.8,\, 8.8]\,\text{kg}$ (+45%), cornering stiffness variation $C_{\alpha f}$, $C_{\alpha r} \in [10,\, 24]\,\text{N/rad}$ (±50%), and steering time constant $\tau_\delta \in [0.05,\, 0.10]\,\text{s}$. The mass range covers payload changes, while the cornering stiffness range accounts for tire wear and surface variation. These uncertainty ranges are sampled via Latin hypercube sampling in the Monte Carlo campaign (Sec. 5).

4 LPV H_∞ Controller Design

This section formulates the generalized plant for H_∞ synthesis, presents the ρ-scheduled weighting functions that encode the bandwidth adaptation strategy, and describes the polytopic synthesis and interpolation procedure.

4.1 Generalized Plant Construction

The mixed-sensitivity H_∞ framework organizes the design around a generalized plant $\mathbf{P}(s, \rho)$ with four signal channels. The exogenous input $\mathbf{w} = [w_{\text{ref}},\, n_\psi,\, n_\delta]^\top$ collects the desired heading from VFG ($w_{\text{ref}} = \psi_{\text{des}}$), the heading sensor noise n_ψ, and the steering encoder noise n_δ. The control input is $u = \delta_{\text{cmd}}$. The performance output $\mathbf{z} = [z_e,\, z_u]^\top$ penalizes the weighted tracking error and actuator effort:

$$z_e = W_e(s, \rho)\, e_\psi, \quad z_u = W_u(s)\, u, \tag{5}$$

where $e_\psi = \psi - \psi_{\text{des}}$ is the heading error. The measurement output $\mathbf{y} = [\psi + n_\psi,\ \delta + n_\delta]^\top$ feeds two noisy sensor signals to the controller. Augmenting the kinematic plant (3) with the weighting functions yields

$$\begin{bmatrix} \mathbf{z} \\ \mathbf{y} \end{bmatrix} = \mathbf{P}(s,\rho) \begin{bmatrix} \mathbf{w} \\ u \end{bmatrix} = \begin{bmatrix} \mathbf{P}_{11} & \mathbf{P}_{12} \\ \mathbf{P}_{21} & \mathbf{P}_{22} \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ u \end{bmatrix}. \tag{6}$$

The design objective is to find a stabilizing controller $\mathbf{K}(s,\rho)$ that minimizes the closed-loop H_∞ norm:

$$\gamma^* = \inf_{\mathbf{K}} \left\| \mathcal{F}_l\big(\mathbf{P}(s,\rho),\, \mathbf{K}(s,\rho)\big) \right\|_\infty, \tag{7}$$

where $\mathcal{F}_l$ is the lower linear fractional transformation.

4.2 Performance Weighting Design

The core idea of this work is to make the error weighting bandwidth a function of $\rho = \kappa v$, so that the controller automatically tightens tracking on curves and relaxes it on straights.

4.2.1 Error weight $W_e(s,\rho)$

A first-order sensitivity weight with ρ-dependent crossover frequency is adopted:

$$W_e(s,\rho) = \frac{s/M_e + \omega_B(\rho)}{s + \omega_B(\rho)\, A_e}, \tag{8}$$

with $A_e = 0.01$ ($1/A_e = 40\,\text{dB}$ steady-state accuracy) and $M_e = 2$ (high-frequency robustness bound). The crossover frequency is scheduled as

$$\omega_B(\rho) = \omega_{B,\min} + \big(\omega_{B,\max} - \omega_{B,\min}\big)\sqrt{\frac{\rho}{\rho_{\max}}} = 5.0 + 4.4\sqrt{\frac{\rho}{5}}, \tag{9}$$

where $\omega_{B,\min} = 5.0\,\text{rad/s}$ and $\omega_{B,\max} = 9.4\,\text{rad/s}$ over $\rho \in [0,\, 5]\,\text{rad/s}$.

The concave square-root shape is chosen for two reasons. First, it allocates more bandwidth growth to the low-ρ region, where the transition from a straight to a mild curve first demands tracking agility. Second, the resulting schedule provides 19% lower heading error at $v = 1.0\,\text{m/s}$ compared with a linear schedule, without degrading high-speed performance.

The lower bound $\omega_{B,\min} = 5.0\,\text{rad/s}$ is set by the kinematic pole. On a straight road ($\rho = 0$), the plant's open-loop pole lies at v_0/L. For speeds up to $v_0 = 1.0\,\text{m/s}$ with $L = 0.2\,\text{m}$, this pole equals $5.0\,\text{rad/s}$, so $\omega_{B,\min} \geq v_0/L$ ensures a minimum loop gain of unity at the kinematic frequency.

Fig. 1 plots the Bode magnitude of W_e at six vertices. As ρ increases from 0 to 5, the crossover shifts from 5.0 to $9.4\,\text{rad/s}$ while the low-frequency gain ($40\,\text{dB}$) and high-frequency bound ($M_e = 2$) remain constant.

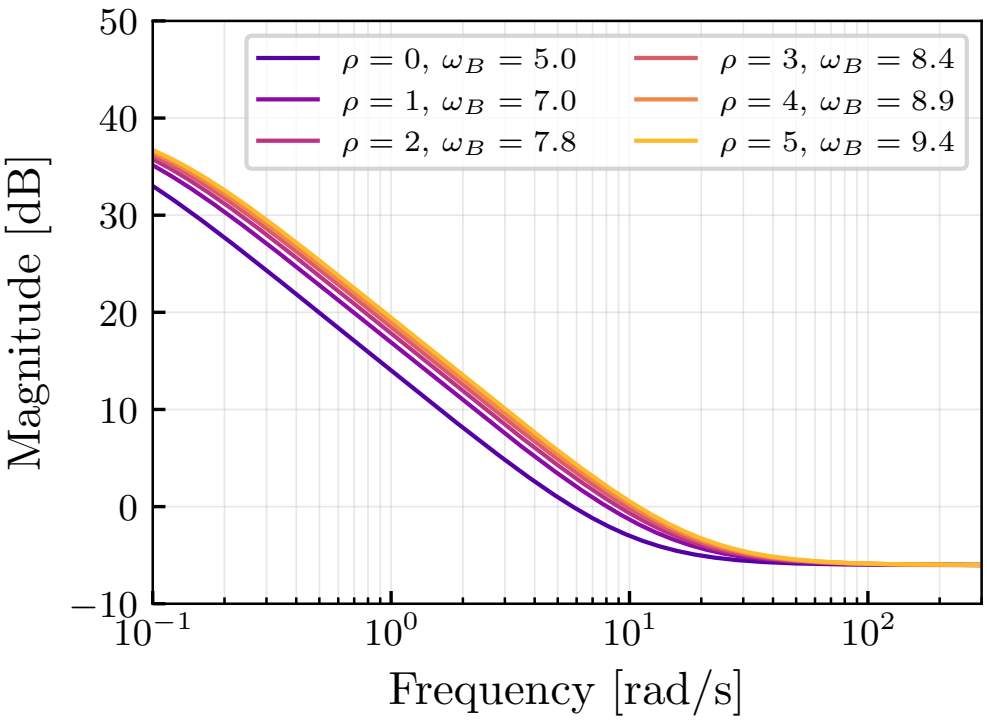

Fig. 1 Bode magnitude of the error weight $W_e(s, \rho)$ at six ρ-vertices. The crossover frequency ω_B increases from 5.0 rad/s ($\rho = 0$) to 9.4 rad/s ($\rho = 5$) following the square-root schedule (9).

Table 2 maps the scheduling parameter to representative driving conditions. At $\rho = 0$ (straight, low speed), the modest bandwidth prioritizes noise rejection. At $\rho = 5$ (sharp curve, high speed), the elevated bandwidth targets rapid heading tracking.

Table 2 Bandwidth Schedule and Driving Conditions

Condition	ρ [rad/s]	ω_B [rad/s]	Priority
Straight, low speed	0	5.0	Noise rejection
Mild curve	2	7.8	Balanced
Sharp curve, high speed	5	9.4	Fast tracking

4.2.2 Control effort weight $W_u(s)$

Actuator usage is penalized by a first-order weight independent of ρ:

$$W_u(s) = \frac{s/M_u + \omega_u}{s + \omega_u/A_u}, \tag{10}$$

with $A_u = 0.1$, $\omega_u = 25.0\,\text{rad/s}$, and $M_u = 5$. The low-frequency gain $1/A_u = 20\,\text{dB}$ moderately limits steady-state control effort, while the high-frequency gain $M_u = 14\,\text{dB}$ enforces strong roll-off beyond ω_u. Because the steering actuator bandwidth ($1/\tau_\delta \approx 14.3\,\text{rad/s}$) is a hardware property unrelated to path geometry, this weight is not scheduled.

4.2.3 Noise weight $W_n(s)$

Sensor noise in both the heading and steering channels is shaped by

$$W_n(s) = \frac{\sigma_n\,(s + \omega_n)}{s + \varepsilon}, \tag{11}$$

where $\sigma_n = 0.02\,\text{rad}$, $\omega_n = 62.8\,\text{rad/s}$ ($\approx 10\,\text{Hz}$), and $\varepsilon = 0.1\,\text{rad/s}$. This high-pass characteristic reflects the broadband nature of the Lidar heading noise and encoder noise described in Sec. 3.3.

4.3 Polytopic Synthesis and Interpolation

4.3.1 Vertex controllers

The range $\rho \in [0,\, 5]$ is sampled at $N = 6$ equally spaced vertices:

$$\mathcal{V} = \{\rho_1 = 0,\ \rho_2 = 1,\ \rho_3 = 2,\ \rho_4 = 3,\ \rho_5 = 4,\ \rho_6 = 5\}. \tag{12}$$

At each vertex ρ_i, the weight $W_e(s, \rho_i)$ is frozen and $\mathbf{P}$ reduces to an LTI system. A standard H_∞ output-feedback synthesis (`hinfsyn` in MATLAB (Zhou et al, 1996)) is solved at each vertex:

$$\mathbf{K}_i = \arg\min_{\mathbf{K}} \gamma_i, \quad \text{s.t.} \quad \|\mathcal{F}_l(\mathbf{P}(s, \rho_i),\, \mathbf{K})\|_\infty < \gamma_i, \quad i = 1, \ldots, 6. \tag{13}$$

Each $\mathbf{K}_i$ is a sixth-order dynamic compensator with realization $(\mathbf{A}_{K,i},\, \mathbf{B}_{K,i},\, \mathbf{C}_{K,i},\, \mathbf{D}_{K,i})$.

4.3.2 Runtime interpolation

At runtime, the measured curvature and speed yield $\rho(t)$, which determines a pair of interpolation weights. For $\rho \in [\rho_j,\, \rho_{j+1}]$:

$$\alpha_j = \frac{\rho_{j+1} - \rho}{\rho_{j+1} - \rho_j}, \quad \alpha_{j+1} = \frac{\rho - \rho_j}{\rho_{j+1} - \rho_j}, \quad \alpha_i = 0 \ \text{ for } i \neq j,\, j{+}1. \tag{14}$$

The controller output is a convex combination of the individual vertex outputs:

$$u(t) = \sum_{i=1}^{N} \alpha_i\big(\rho(t)\big)\, u_i(t), \tag{15}$$

where each u_i is computed by running the i-th vertex controller with its own internal state. This output-interpolation scheme avoids the state-dimension mismatch that can arise when directly blending state-space matrices from different realizations.

Fig. 2 shows the Bode magnitude of the six vertex controllers. The low-frequency gain increases with ρ, reflecting the higher tracking demand, while the high-frequency roll-off imposed by W_u remains consistent across all vertices.

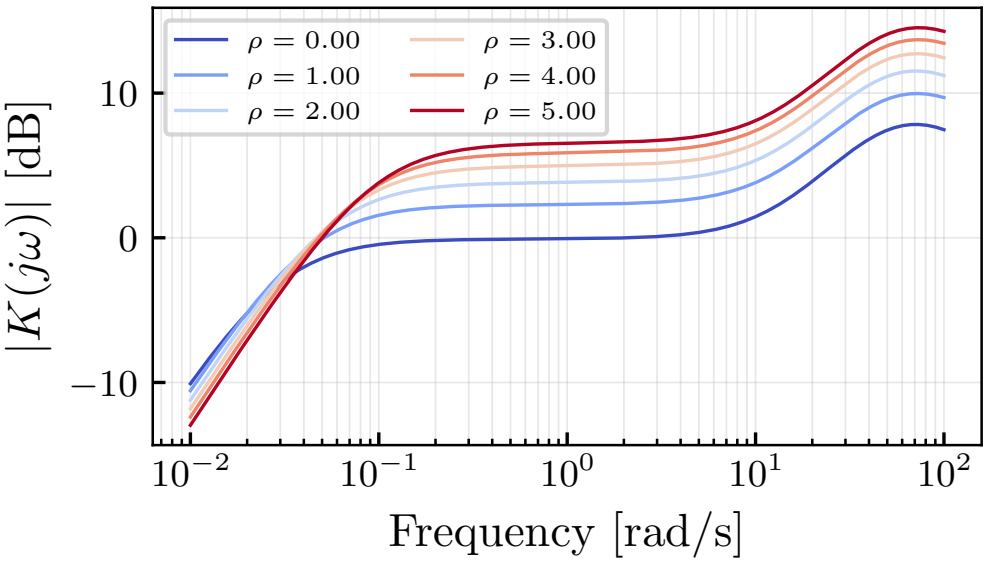

Fig. 2 Bode magnitude of the six vertex controllers $\mathbf{K}_i(j\omega)$ for $\rho_i = 0, \ldots, 5$. Higher ρ yields higher low-frequency gain to match the increased guidance bandwidth.

4.4 Stability and Performance

A common Lyapunov matrix $\mathbf{X} \succ 0$ satisfying the standard polytopic stability conditions at all six vertices was found numerically, following the methodology of Apkarian et al (1995). This guarantees quadratic stability for all admissible ρ trajectories within the convex hull of $\mathcal{V}$, including arbitrarily fast parameter variations (Briat, 2015). All six vertex syntheses achieve $\gamma_i < 1$, so the overall LPV performance bound is

$$\gamma_{\text{LPV}} \leq \max_{i=1,\ldots,N} \gamma_i < 1, \tag{16}$$

confirming that the weighted sensitivity and control-sensitivity specifications are satisfied at every operating point.

4.5 Frequency-Domain Verification

The preceding subsection confirmed stability via a common Lyapunov matrix. This subsection provides additional design verification in the frequency domain.

Fig. 3 plots the sensitivity function $|S(j\omega)|$ at all six ρ-vertices together with the PID-FF sensitivity. The peak sensitivity remains below 6 dB at every vertex (4.3 dB at $\rho = 0$, 5.7 dB at $\rho = 5$), satisfying the classical robustness criterion $\|S\|_\infty < 2$ (Skogestad and Postlethwaite, 2005). The corresponding modulus margins range from 0.52 to 0.61, meaning the Nyquist contour maintains a safe distance from the critical point -1 throughout the scheduling range.

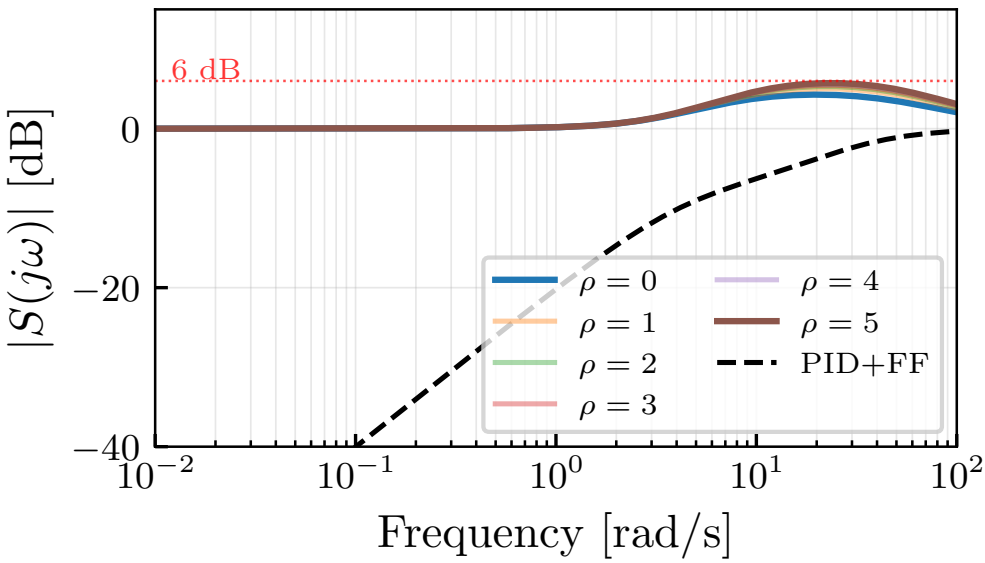

Fig. 3 Sensitivity function $|S(j\omega)|$ at six ρ-vertices and PID-FF. All vertices satisfy the 6 dB robustness bound.

Table 3 summarizes the peak sensitivity at each vertex. The monotonic increase of $\|S\|_\infty$ with ρ reflects the higher bandwidth demand: as the error weight W_e pushes the crossover frequency upward, the controller achieves tighter tracking at the cost of a marginally reduced robustness margin, yet the 6 dB ceiling is never breached.

Fig. 4 compares the open-loop transfer function $L(j\omega)$ for three selected vertices ($\rho = 0, 2, 5$) and PID-FF. At $\rho = 0$ (straight), the LPV loop gain is lower than PID-FF in the mid-frequency range, reducing amplification of sensor noise. As ρ increases, the loop gain rises substantially from the $\rho = 0$ baseline; at $\rho = 5$ (sharp curve at high speed), it reaches a level comparable to PID-FF in the tracking band (0.2–3 rad/s) while maintaining a steeper high-frequency roll-off that limits noise sensitivity. This adaptive loop-shaping is the frequency-domain mechanism underlying the speed-invariant time-domain performance observed in Sec. 5.

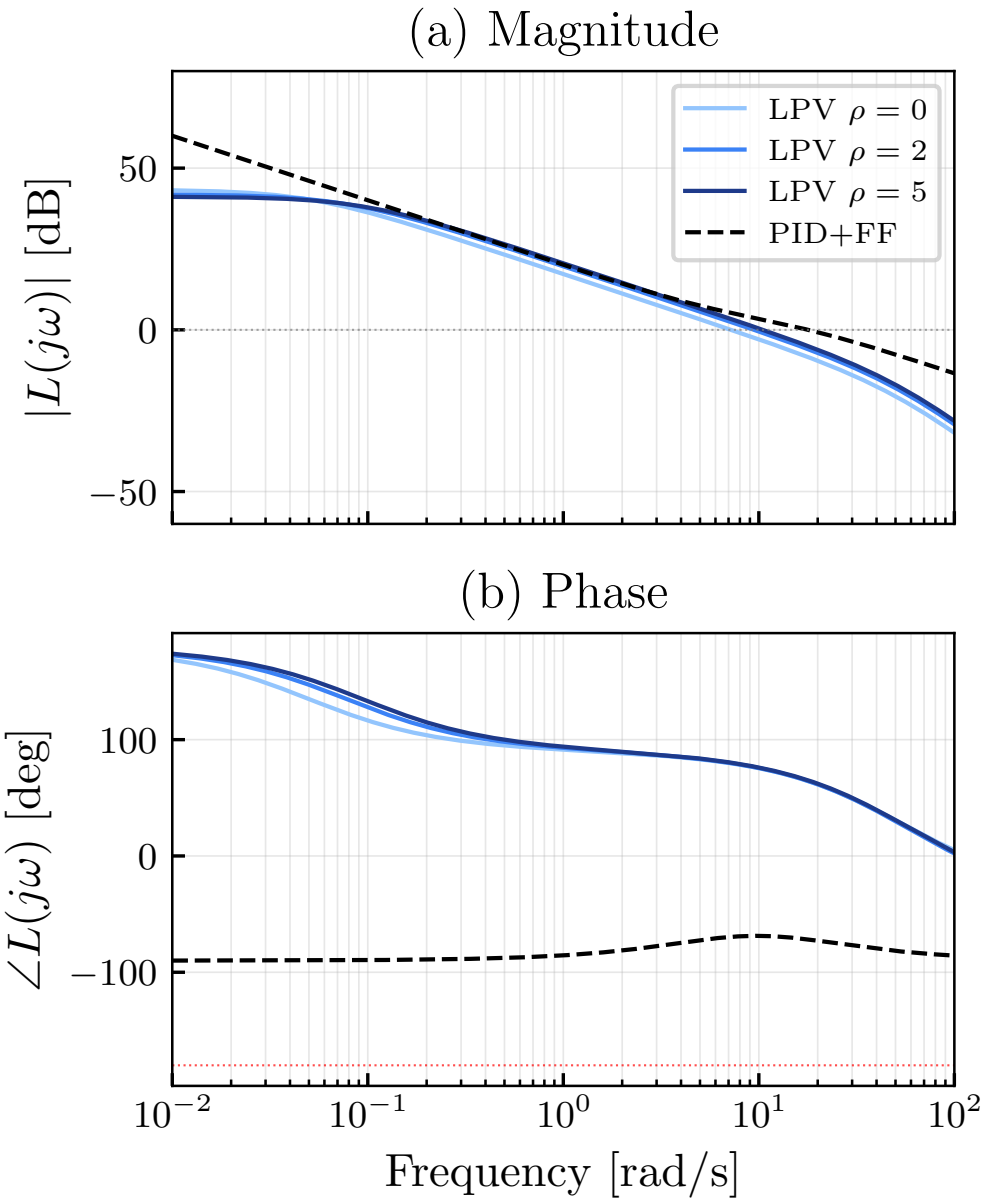

Fig. 4 Open-loop Bode plot of $L(j\omega)$: LPV-H_∞ at $\rho = 0, 2, 5$ versus PID-FF. Higher ρ increases the loop gain to match the guidance bandwidth demand.

Table 3 Frequency-Domain Robustness Metrics at Each ρ-Vertex

Vertex	ρ [rad/s]	$\|S\|_\infty$ [dB]	Modulus margin
$\mathbf{K}_1$	0	4.3	0.61
$\mathbf{K}_2$	1	5.0	0.56
$\mathbf{K}_3$	2	5.3	0.54
$\mathbf{K}_4$	3	5.5	0.53
$\mathbf{K}_5$	4	5.6	0.53
$\mathbf{K}_6$	5	5.7	0.52
PID-FF	—	-0.3	—

4.6 Implementation Considerations

The runtime LPV controller consists of a simple interpolation between two adjacent vertex controllers, outlined in Algorithm 1.

Algorithm 1 Runtime LPV Interpolation

Require: $\rho(t)$, vertex controllers $\mathbf{K}_1, \ldots, \mathbf{K}_N$
Ensure: $u(t)$

```
j ← ⌊ρ⌋, clamp to [0, N−2]
α ← ρ − j
u_j ← K_j.output(y) {matrix-vector multiply}
u_{j+1} ← K_{j+1}.output(y)
u ← (1 − α) u_j + α u_{j+1}
return u
```

Each step requires two matrix-vector multiplications (one per bracketing vertex) and one scalar interpolation. Table 4 reports the per-step wall-clock time measured over 1,000 iterations on a desktop CPU (Apple M4 Pro, single thread).

Table 4 Per-Step Computation Time (1,000 iterations)

Controller	Mean [μs]	Memory [bytes]
LPV-H_∞	10.8	2,688
Linear MPC	24.4	—
PID+FF	1.4	40

The proposed LPV controller runs at approximately 11 μs per step, which is well within real-time constraints for the 10 ms control period ($T_s = 0.01$ s) used in the simulation. The memory footprint is $6 \times (36 + 12 + 6 + 2) \times 8 = 2{,}688$ bytes (six vertex state-space matrices stored in double precision), small enough for deployment on embedded platforms. PID-FF is the lightest at 1.4 μs, while linear MPC with pre-compiled QP (Stellato et al, 2020) requires 24 μs—still fast, but with higher memory overhead from the QP matrices.

5 Simulation Study

This section evaluates the proposed ρ-scheduled LPV-H_∞ controller through deterministic scenario tests and a Monte Carlo robustness campaign. Two path scenarios exercise distinct curvature profiles, and three controllers are compared across a speed range of 0.5–3.0 m/s.

5.1 Simulation Setup

Controller synthesis uses MATLAB R2024b (`hinfsyn` from the Robust Control Toolbox). The resulting state-space matrices at each polytopic vertex are exported in JSON format and loaded into a Python 3.12 simulation environment. Closed-loop time-domain integration is performed with `scipy.integrate.solve_ivp` (explicit Runge–Kutta RK45, maximum step size 0.001 s).

Three controllers are compared:

1. **LPV-H_∞ (proposed)**: Polytopic controller scheduled by $\rho = |\kappa|v$ with the bandwidth law $\omega_B(\rho)$ defined in (9). Six vertex controllers are interpolated online.
2. **Linear MPC**: Prediction horizon $N_p = 20$, control horizon $N_c = 5$, sampling period $T_s = 0.01$ s. The quadratic program is solved with OSQP (Stellato et al, 2020).
3. **PID + feedforward (PID-FF)**: $\delta_{\text{cmd}} = K_P e_\psi + K_D \dot{e}_\psi + \arctan(L\kappa)$, where the feedforward term compensates steady-state curvature following (Snider, 2009).

All three controllers share the same VFG guidance layer with gain $k_e = 3.0$, so any performance difference originates exclusively from the control law. Steering actuation is modeled as a first-order lag with time constant $\tau_\delta = 0.07$ s. All three controllers enforce the steering saturation limit $|\delta_{\text{cmd}}| \leq 0.5$ rad at the controller output.

Four metrics quantify performance: (i) RMS heading error $e_{\psi,\text{RMS}}$, (ii) maximum lateral error $e_{y,\max}$, (iii) RMS steering rate $\dot{\delta}_{\text{RMS}}$, and (iv) settling time t_s (time after which $|e_\psi| < 0.02$ rad).

Fig. 5 illustrates the geometry of the two test scenarios.

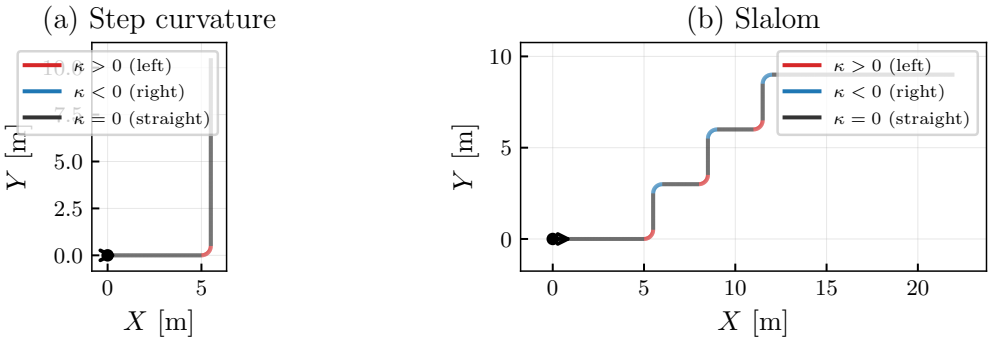

Fig. 5 Reference path geometry for (a) step curvature and (b) slalom scenarios. The step curvature path transitions abruptly from $\kappa = 0$ to $\kappa = 2\,\mathrm{m}^{-1}$; the slalom path alternates between $\kappa = \pm 2\,\mathrm{m}^{-1}$ through six arcs.

5.2 Scenario A: Step Curvature

The reference path begins with a 5 m straight segment, transitions into a 90° circular arc of radius $R = 0.5\,\mathrm{m}$ ($\kappa = 2\,\mathrm{m}^{-1}$), and returns to a straight run-out. This profile creates a step change in the scheduling parameter from $\rho = 0$ on the straight to $\rho = \kappa v$ on the arc, testing how quickly each controller adapts to the bandwidth demand.

Six constant speeds are tested: $v \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}\,\mathrm{m/s}$, yielding $\rho_{\mathrm{arc}} \in \{1.0, 2.0, 3.0, 4.0, 5.0, 6.0\}$. At $v = 3.0\,\mathrm{m/s}$, $\rho = 6.0$ exceeds the synthesis range ($\rho_{\max} = 5$); the controller saturates at the uppermost vertex yet remains stable. The kinematic bicycle model with sensor noise ($\sigma_\psi = 0.03\,\mathrm{rad}$, $\sigma_\delta = 0.015\,\mathrm{rad}$) is used.

Table 5 Scenario A: Step Curvature — RMS Heading Error [deg]

v_0 [m/s]	PID+FF	Linear MPC	LPV-H_∞
0.5	10.88	**2.59**	4.40
1.0	7.93	**2.64**	2.90
1.5	6.93	3.04	**2.54**
2.0	5.54	3.47	**2.55**
2.5	5.21	4.42	**3.44**
3.0	5.67	4.49	**2.85**

Table 5 reports the RMS heading error across all speeds. At low speed ($v \leq 1.0\,\mathrm{m/s}$), MPC produces the smallest error because the guidance bandwidth remains low and a fixed-gain controller can track the slowly varying reference without difficulty. As speed increases, the proposed LPV-H_∞ controller takes the lead. At $v = 1.5\,\mathrm{m/s}$ it achieves 2.54°, surpassing MPC (3.04°) by 16%. The gap widens at $v = 2.0\,\mathrm{m/s}$: the LPV-H_∞ error is 2.55°, compared with 3.47° for MPC (27% reduction) and 5.54° for PID-FF (54% reduction). At $v = 2.5$ and $3.0\,\mathrm{m/s}$ the advantage persists, confirming that the ρ-scheduled bandwidth adapts correctly even when ρ exceeds the design polytope.

Fig. 6 presents the time histories at $v = 2.0\,\mathrm{m/s}$. The heading error of MPC and PID-FF peaks sharply at the curvature onset, whereas the LPV-H_∞ controller exhibits a noticeably smaller overshoot and faster convergence on the subsequent straight. The bottom panel shows the scheduling parameter ρ rising to 4.0 during the arc and returning to zero on the straight, which drives the real-time bandwidth adaptation.

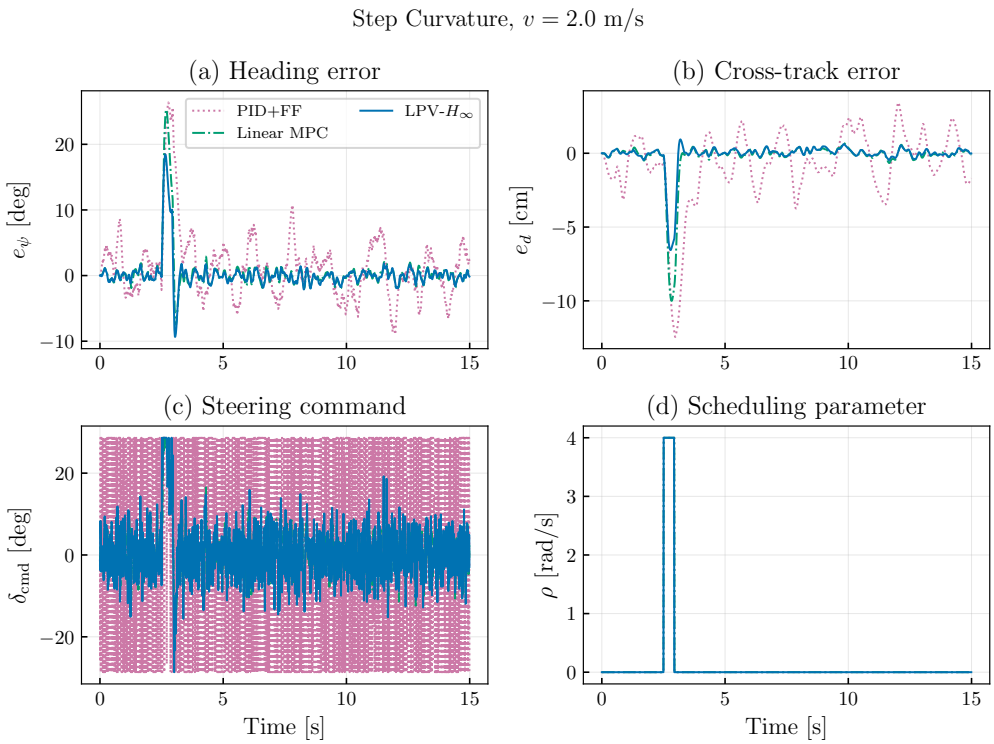

Fig. 6 Scenario A at $v = 2.0\,\mathrm{m/s}$: (a) heading error, (b) cross-track error, (c) steering command, (d) scheduling parameter ρ. The LPV-H_∞ controller adapts its bandwidth with ρ, producing the smallest overshoot and fastest recovery.

Fig. 7 zooms into the curvature onset region at $v = 2.0\,\mathrm{m/s}$. At the moment the path transitions from straight to curved ($\kappa = 0 \rightarrow 2\,\mathrm{m}^{-1}$), the LPV-$H_\infty$ controller responds with a faster steering command build-up because ρ jumps to 4.0 and the bandwidth rises to $\omega_B \approx 9.0\,\mathrm{rad/s}$. MPC and PID-FF, operating at fixed bandwidth, exhibit larger overshoot and slower recovery in this transient window.

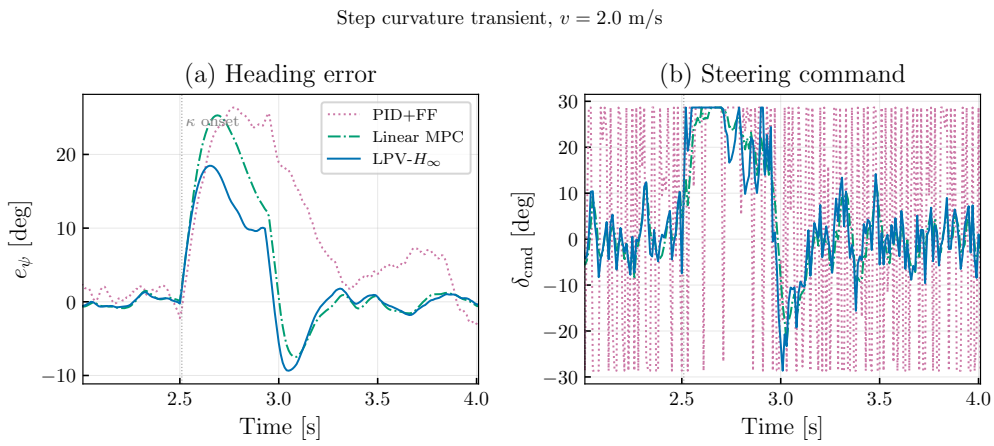

Fig. 7 Transient response at curvature onset ($v = 2.0$ m/s): (a) heading error, (b) steering command. The LPV-H_∞ controller reacts faster due to instantaneous bandwidth adaptation.

5.3 Scenario B: Slalom

The slalom path alternates between left and right circular arcs ($R = 0.5$ m, subtended angle $\theta = \pi/2$) connected by short straight transitions, covering approximately 45 m in total. The curvature switches between $\kappa = +2$ and $\kappa = -2\,\mathrm{m}^{-1}$ at each arc boundary, so the path exercises repeated sign reversals without the closest-point ambiguity that arises in self-intersecting trajectories such as a figure-eight.

Table 6 Scenario B: Slalom — RMS Heading Error [deg]

v_0 [m/s]	PID+FF	Linear MPC	LPV-H_∞
1.0	14.36	**5.30**	5.45
1.5	14.63	7.31	**5.52**
2.0	10.93	8.74	**5.61**
2.5	11.81	10.28	**6.20**
3.0	11.49	12.42	**6.70**

Table 6 summarizes the results across five speeds ($v = 1.0$–3.0 m/s); the lowest speed $v = 0.5$ m/s is excluded because the vehicle traverses only a single arc within the simulation window, preventing the alternating curvature reversals that define the slalom. The LPV-H_∞ controller achieves the lowest RMS heading error from $v = 1.5$ m/s onward; at $v = 1.0$ m/s, MPC marginally leads (5.30° vs. 5.45° for LPV-H_∞). At $v = 2.0$ m/s, LPV-H_∞ records 5.61° versus 8.74° for MPC (36% reduction) and 10.93° for PID-FF (49% reduction). The contrast at $v = 3.0$ m/s is especially pronounced: LPV-H_∞ achieves 6.70°, while MPC reaches 12.42° (46% reduction) and PID-FF reaches 11.49° (42% reduction).

The key observation is how differently the errors scale with speed. MPC and PID-FF errors grow rapidly because the repeated curvature reversals generate guidance-rate demands that exceed the capability of a fixed-bandwidth controller. In contrast, the LPV-H_∞ error remains nearly flat across $v = 1.5$–$3.0\,\mathrm{m/s}$ (5.52–6.70°), because the scheduling variable $\rho = |\kappa| v$ increases proportionally with speed and pushes ω_B toward $\omega_{B,\max}$, matching the bandwidth to the guidance demand at each operating point.

Fig. 8 shows the time series at $v = 2.5\,\mathrm{m/s}$. The heading error of MPC and PID-FF grows progressively at each successive curvature reversal, whereas the LPV-H_∞ controller keeps the oscillation bounded throughout the entire maneuver. The scheduling parameter ρ alternates between 0 and 5.0, exercising the full polytopic range at every arc transition.

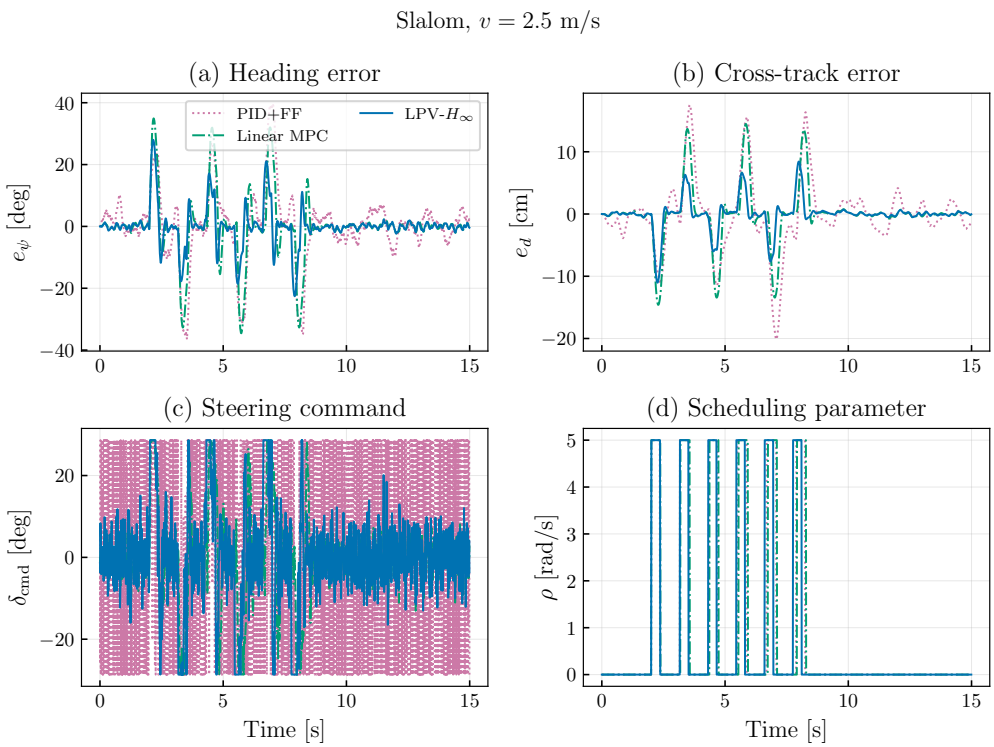

Fig. 8 Scenario B at $v = 2.5\,\mathrm{m/s}$: (a) heading error, (b) cross-track error, (c) steering command, (d) scheduling parameter ρ. Despite repeated curvature sign changes, the LPV-H_∞ controller maintains bounded oscillation.

5.4 Speed-Performance Analysis

Fig. 9 plots the RMS heading error as a continuous function of speed for both scenarios. To quantify speed invariance, we define the Speed Invariance index

$$\mathrm{SI} = \frac{e_{\psi,\max} - e_{\psi,\min}}{e_{\psi,\mathrm{mean}}}, \tag{17}$$

where the statistics are computed over $v \in \{1.0, 1.5, 2.0, 2.5, 3.0\}$ m/s.

For step curvature, the LPV-H_∞ error varies only between 2.54° and 3.44° (SI = 0.32), while MPC spans 2.64–4.49° (SI = 0.51) and PID-FF spans 5.21–7.93° (SI = 0.44). On the slalom, LPV-H_∞ achieves SI = 0.21 (5.45–6.70°) versus SI = 0.81 for MPC (5.30–12.42°) and SI = 0.30 for PID-FF (10.93–14.63°).

The low SI of LPV-H_∞ confirms that ρ-scheduling compensates the speed-dependent guidance bandwidth. This is consistent with the frequency-domain analysis of Sec. 4.5: as ρ increases with speed, the loop gain rises to maintain tracking performance.

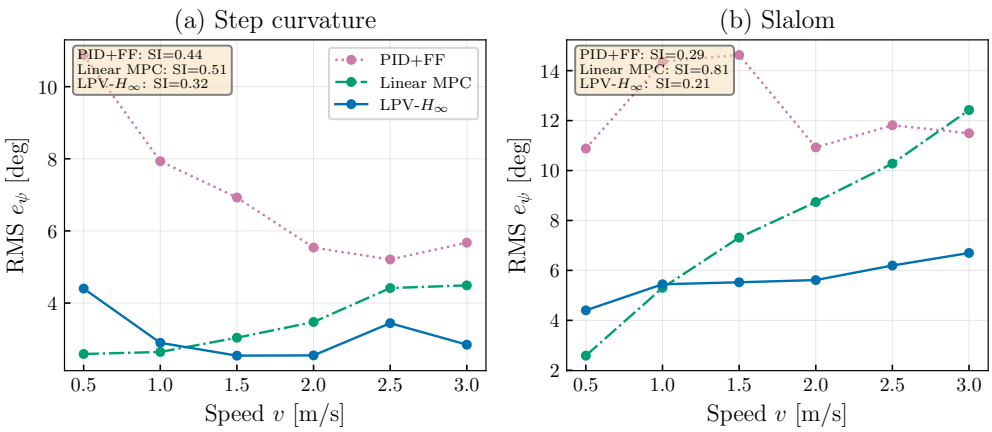

Fig. 9 RMS heading error versus speed for (a) step curvature and (b) slalom. The LPV-H_∞ curve remains nearly flat, while MPC and PID-FF degrade at higher speeds. Speed Invariance (SI) indices are annotated.

5.5 Monte Carlo Robustness Analysis

The deterministic tests above use a kinematic model with known parameters. To evaluate robustness against plant–model mismatch, a Monte Carlo campaign is conducted with the dynamic bicycle model (4) and perturbed parameters.

5.5.1 Protocol

A total of 200 parameter samples are generated via Latin Hypercube Sampling (LHS) (McKay et al, 1979) over a five-dimensional uncertainty space:

- Mass: $m \in [4.8, 8.8]$ kg.
- Yaw inertia: $I_z \in [0.10, 0.25]$ kg·m^2.
- Front cornering stiffness: $C_{\alpha f} \in [10, 24]$ N/rad.
- Rear cornering stiffness: $C_{\alpha r} \in [10, 24]$ N/rad.
- Steering time constant: $\tau_\delta \in [0.05, 0.10]$ s.

Sensor noise is fixed at $\sigma_\psi = 0.03$ rad and $\sigma_\delta = 0.015$ rad. The test scenario is Step Curvature (Scenario A) at four speeds: $v \in \{1.0, 1.5, 2.0, 2.5\}$ m/s, giving $200 \times 4 \times 3 =$ 2,400 simulation runs in total. A run is classified as unstable if the RMS heading error computed over the final 20% of the simulation time exceeds 30°.

5.5.2 Stability

All three controllers achieve a 200/200 stability rate (100%) at every tested speed. For the proposed LPV-H_∞ controller, this result is consistent with the quadratic stability property guaranteed by the common Lyapunov function across the polytope: the stability certificate does not depend on the plant parameters, so the controller tolerates the perturbed dynamics without loss of stability.

5.5.3 Performance distribution

Table 7 and Fig. 10 present the statistical summary and box-plot distributions, respectively.

Table 7 Monte Carlo Performance Statistics (200 LHS Runs; All Controllers Achieve 100% Stability)

v_0 [m/s]	Controller	Median [deg]	Mean ± Std [deg]
1.0	PID+FF	7.27	7.30 ± 0.87
	Linear MPC	**2.74**	2.75 ± 0.25
	LPV-H_∞	2.97	2.98 ± 0.13
1.5	PID+FF	6.54	6.54 ± 0.80
	Linear MPC	3.17	3.23 ± 0.42
	LPV-H_∞	**2.66**	2.73 ± 0.33
2.0	PID+FF	5.93	6.02 ± 0.76
	Linear MPC	3.77	3.85 ± 0.57
	LPV-H_∞	**2.89**	3.00 ± 0.56
2.5	PID+FF	5.91	5.97 ± 0.72
	Linear MPC	4.50	4.56 ± 0.73
	LPV-H_∞	**3.29**	3.49 ± 0.83

At $v = 1.0$ m/s, MPC achieves the lowest median RMS heading error (2.74°), with LPV-H_∞ close behind (2.97°). The roles reverse at $v = 1.5$ m/s: LPV-H_∞ attains 2.66°, outperforming MPC (3.17°) by 16%. At $v = 2.0$ m/s the gap grows to 23% (LPV-H_∞: 2.89°; MPC: 3.77°), and at $v = 2.5$ m/s it reaches 27% (LPV-H_∞: 3.29°; MPC: 4.50°). PID-FF lags all controllers with a median exceeding 5.9° at every speed.

A notable feature is the speed sensitivity of each controller. The MPC median error increases from 2.74° to 4.50° as speed rises from 1.0 to 2.5 m/s—a 64% degradation. PID-FF shows a similar trend (7.27° → 5.91°, though non-monotonic due to curvature feedforward saturation effects). The LPV-H_∞ median, by contrast, remains nearly constant across all four speeds (2.66–3.29°), confirming that the ρ-scheduling effectively compensates for the speed-dependent guidance bandwidth.

The standard deviation of LPV-H_∞ ranges from 0.13 to 0.83°, compared with 0.25–0.73° for MPC and 0.72–0.87° for PID-FF. At $v \leq 2.0$ m/s, LPV-H_∞ exhibits the narrowest spread; at $v = 2.5$ m/s the LPV and MPC dispersions are comparable (0.83 vs. 0.73°). Overall, the H_∞-optimal design provides consistent performance despite large parametric perturbations in the plant.

To verify statistical significance, Wilcoxon signed-rank tests are applied to each pairwise comparison at every speed. All comparisons between LPV-H_∞ and each baseline yield $p < 10^{-10}$, confirming that the observed differences are not attributable to sampling variability.

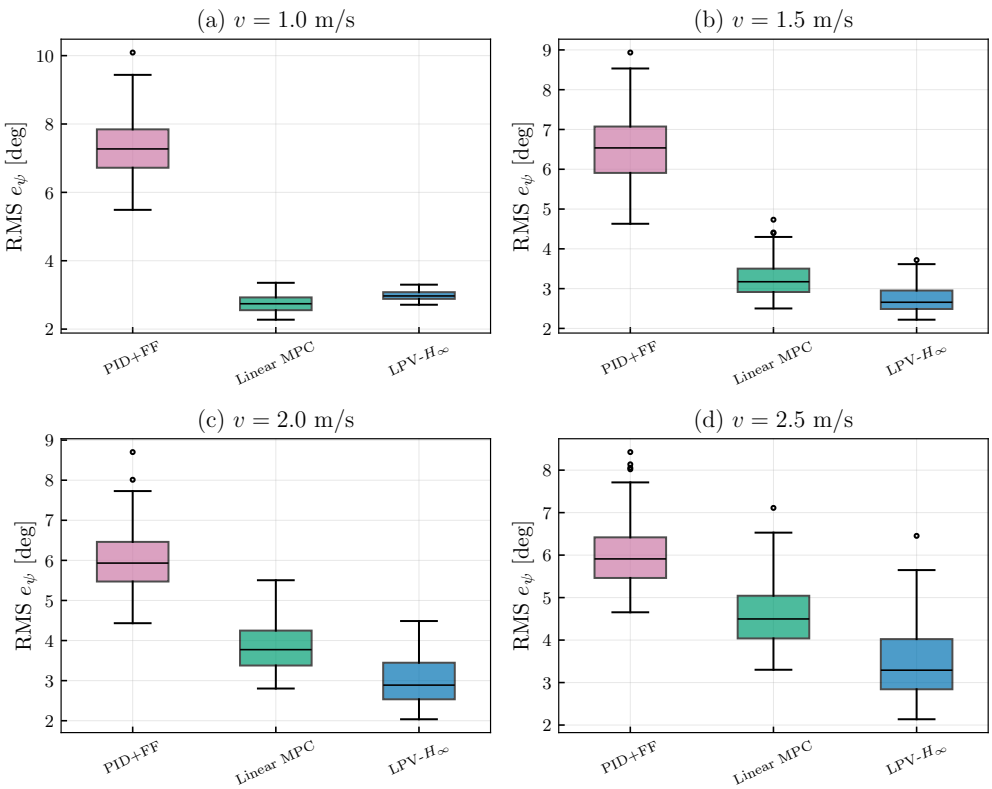

Fig. 10 Monte Carlo box plots of RMS heading error (200 LHS samples per speed). The LPV-H_∞ controller achieves the lowest median at $v \geq 1.5$ m/s and maintains a nearly speed-invariant error distribution.

5.5.4 Settling time distribution

Fig. 11 shows the settling time distributions across the 200 MC samples. At $v = 1.0\,\mathrm{m/s}$, the LPV-H_∞ controller achieves a median settling time of 9.7 s, compared with 14.2 s for MPC and 15.0 s for PID-FF. The 1.5× advantage over MPC at this low speed is notable because the RMS error comparison is nearly tied (2.97° vs. 2.74°): the LPV controller reaches steady state substantially faster, even when the asymptotic error is similar. At higher speeds, settling times converge because all controllers quickly enter the curved section where steady-state tracking dominates.

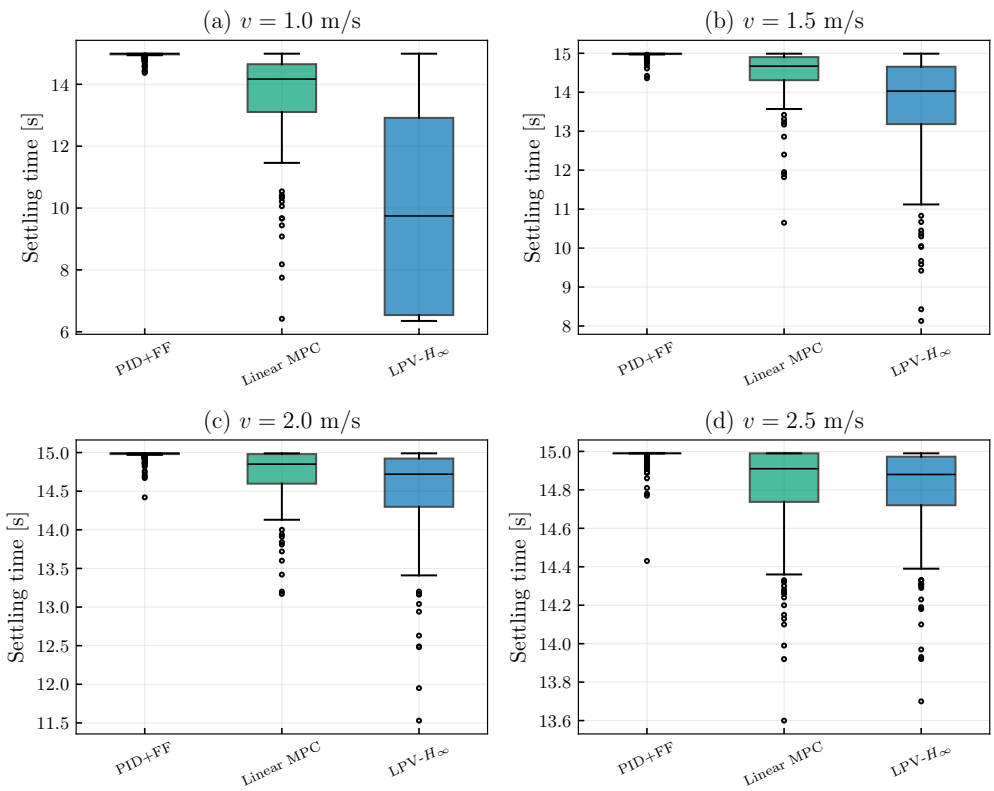

Fig. 11 Monte Carlo box plots of settling time (200 LHS samples per speed). At $v = 1.0\,\mathrm{m/s}$, the LPV-H_∞ controller settles 1.5× faster than MPC.

6 Conclusion

This paper presented an LPV H_∞ path-following controller that schedules its performance bandwidth according to the curvature-velocity product $\rho = \kappa v$. By exploiting

the structural relationship between vector field guidance bandwidth and the instantaneous path-speed condition, the proposed controller achieves adaptive tracking aggressiveness without requiring online optimization or runtime gain tuning.

The key contribution is the demonstration that a simple, physics-motivated scheduling variable can effectively decouple tracking performance from operating speed. Comparative simulations confirmed that the proposed controller consistently outperforms both linear MPC and PID with curvature feedforward at moderate to high speeds, while maintaining comparable performance at low speeds where the guidance demand is modest. The near speed-invariant error profile—in contrast to the monotonic degradation exhibited by the baselines—validates the frequency-domain design rationale linking ρ-scheduled bandwidth to guidance demand. Monte Carlo robustness analysis under parametric uncertainty further established that these advantages are preserved across a broad range of plant conditions, with statistically significant improvements at all tested speeds.

Three directions for future work are envisioned. First, hardware validation on a physical scale-car platform will assess real-world robustness under unmodeled dynamics and sensor imperfections. Second, a multi-parameter scheduling framework incorporating longitudinal speed as an independent variable may improve transient response in low-curvature segments where ρ alone provides limited scheduling authority. Third, curvature preview-based scheduling that leverages look-ahead path geometry could enable proactive bandwidth adjustment before curvature transitions occur.

Acknowledgments. The author used Claude (Anthropic) for English language editing of the manuscript. The author reviewed and revised all AI-generated suggestions and takes full responsibility for the content of the publication.

Declarations

- **Funding** This work was supported by the BK21 Four Program (2120240815267) of the National Research Foundation of Korea (NRF) funded by the Ministry of Education, Korea.
- **Conflict of interest** The author declares no conflict of interest.
- **Data availability** The simulation data and code that support the findings of this study are available from the corresponding author upon reasonable request.

References

Aguiar AP, Hespanha JaP (2007) Trajectory-tracking and path-following of underactuated autonomous vehicles with parametric modeling uncertainty. IEEE Transactions on Automatic Control 52(8):1362–1379. https://doi.org/10.1109/TAC.2007.902731

Apkarian P, Gahinet P, Becker G (1995) Self-scheduled $\mathcal{H}_\infty$ control of linear parameter-varying systems: A design example. Automatica 31(9):1251–1261. https://doi.org/10.1016/0005-1098(95)00038-X

Briat C (2015) Linear Parameter-Varying and Time-Delay Systems: Analysis, Observation, Filtering & Control, Advances in Delays and Dynamics, vol 3. Springer, https://doi.org/10.1007/978-3-662-44050-6

Corno M, Panzani G, Roselli F, et al (2021) An LPV approach to autonomous vehicle path tracking in the presence of steering actuation nonlinearities. IEEE Transactions on Control Systems Technology 29(4):1766–1774. https://doi.org/10.1109/TCST.2020.3006123

Falcone P, Borrelli F, Asgari J, et al (2007) Predictive active steering control for autonomous vehicle systems. IEEE Transactions on Control Systems Technology 15(3):566–580. https://doi.org/10.1109/TCST.2007.894653

Gáspár P, Szászi I, Bokor J (2005) Reconfigurable control structure to prevent the rollover of heavy vehicles. Control Engineering Practice 13(6):699–711. https://doi.org/10.1016/j.conengprac.2004.06.003

Gonçalves VM, Pimenta LCA, Maia CA, et al (2010) Vector fields for robot navigation along time-varying curves in n-dimensions. IEEE Transactions on Robotics 26(4):647–659. https://doi.org/10.1109/TRO.2010.2053077

Kapitanyuk YA, Proskurnikov AV, Cao M (2018) A guiding vector-field algorithm for path-following control of nonholonomic mobile robots. IEEE Transactions on Control Systems Technology 26(4):1372–1385. https://doi.org/10.1109/TCST.2017.2705059

Kapsalis D, Sename O, Milanés V, et al (2022) A reduced LPV polytopic look-ahead steering controller for autonomous vehicles. Control Engineering Practice 129:105360. https://doi.org/10.1016/j.conengprac.2022.105360

Lee S (2023) Vector field-based guidance in space based on implicitization of parametric curves using approximate orthogonal surfaces. IFAC-PapersOnLine 56(2):1160–1167. https://doi.org/10.1016/j.ifacol.2023.10.1722, 22nd IFAC World Congress

McKay MD, Beckman RJ, Conover WJ (1979) A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. Technometrics 21(2):239–245. https://doi.org/10.1080/00401706.1979.10489755

Nelson DR, Barber DB, McLain TW, et al (2007) Vector field path following for miniature air vehicles. IEEE Transactions on Robotics 23(3):519–529. https://doi.org/10.1109/TRO.2007.898976

Rajamani R (2012) Vehicle Dynamics and Control, 2nd edn. Mechanical Engineering Series, Springer, https://doi.org/10.1007/978-1-4614-1433-9

Sename O, Medero Borrell A, Menezes Morato M (2024) On robust and predictive control approaches for linear parameter varying systems: Application to vehicle lateral control. Foundations and Trends in Systems and Control 11(4):285–380. https://doi.org/10.1561/2600000032

Skogestad S, Postlethwaite I (2005) Multivariable Feedback Control: Analysis and Design, 2nd edn. Wiley, Chichester, UK

Snider JM (2009) Automatic steering methods for autonomous automobile path tracking. Tech. Rep. CMU-RI-TR-09-08, Robotics Institute, Carnegie Mellon University, Pittsburgh, PA

Stellato B, Banjac G, Goulart P, et al (2020) OSQP: An operator splitting solver for quadratic programs. Mathematical Programming Computation 12(4):637–672. https://doi.org/10.1007/s12532-020-00179-2

Sujit PB, Saripalli S, Sousa JaB (2014) Unmanned aerial vehicle path following: A survey and analysis of algorithms for fixed-wing unmanned aerial vehicles. IEEE Control Systems Magazine 34(1):42–59. https://doi.org/10.1109/MCS.2013.2287568

Zhou K, Doyle JC, Glover K (1996) Robust and Optimal Control. Prentice Hall, Upper Saddle River, NJ